

D2C Customer Churn Prediction API

Project Overview

This repository contains Part 4 of the D2C Customer Churn Capstone Project. It provides a FastAPI-based microservice that serves the Random Forest churn prediction model trained in Part 3. The API accepts customer behavioral and transactional data, evaluates it against the model, and returns a churn probability, a predicted class (based on a business-optimized 0.35 threshold), and a dynamic risk explanation.

Repository Structure

- `app/main.py`: The FastAPI application containing the endpoints, Pydantic schemas, and inference logic.
- `train_model.py`: A bootstrap script to generate a synthetic dataset and train/save `model.pkl` so the API can run immediately out of the box.
- `tests/test_api.py`: Pytest suite covering input validation, health checks, and prediction endpoints.
- `monitoring_plan.md`: Documentation on tracking model drift, API health, and responsible AI use guidelines.
- `Dockerfile & requirements.txt`: Environment and reproducibility configuration.

Setup Instructions

Option 1: Local Python Environment

1. **Install Dependencies:**
`pip install -r requirements.txt`
2. **Generate the Model Artifact:** *(Run this once to create `model.pkl`)*
`python train_model.py`
3. **Run the API Server:**
`uvicorn app.main:app --reload`

The API will be accessible at <http://127.0.0.1:8000>. You can view the interactive documentation at <http://127.0.0.1:8000/docs>.

Option 2: Docker Setup (Recommended)

1. **Build the image:**
`docker build -t churn-api .`
2. **Run the container:**

```
docker run -p 8000:8000 churn-api
```

Test Execution Instructions

To run the automated API test suite:

```
# Ensure you are in the project root
pytest tests/test_api.py -v
```

API Endpoint Details

1. GET /health

Returns the health status of the API and verifies if the model is loaded.

Sample Response:

```
{
  "status": "ok",
  "model_loaded": true
}
```

2. POST /predict

Evaluates a single customer.

Sample Request:

```
{
  "city_tier": "Tier 2",
  "age_group": "25-34",
  "acquisition_channel": "Instagram",
  "loyalty_tier": "Silver",
  "preferred_category": "Skin Care",
  "marketing_consent": "Yes",
  "recency_days": 110,
  "frequency_180d": 2,
  "monetary_180d": 450.50,
  "return_rate_180d": 0.0,
  "avg_discount_pct_180d": 0.15,
  "avg_rating_180d": 4.0,
  "category_diversity_180d": 2,
  "ticket_count_90d": 1,
```

```
"negative_ticket_rate_90d": 1.0,  
"avg_resolution_hours_90d": 48.5,  
"days_since_signup": 200,  
"sessions_30d": 1,  
"product_views_30d": 3,  
"cart_adds_30d": 0,  
"wishlist_adds_30d": 0,  
"abandoned_carts_30d": 0,  
"email_opens_30d": 0,  
"campaign_clicks_30d": 0,  
"last_visit_days_ago": 25  
}
```

Sample Response:

```
{  
  "churn_probability": 0.68,  
  "predicted_class": 1,  
  "risk_level": "high",  
  "risk_explanation": "Model predicts a 68% chance of churn due to: high recency (lack of recent  
orders), negative support experience, low recent web engagement."  
}
```

3. POST /batch_predict

Accepts a list of customer objects (same schema as above) and returns an array of predictions.